便携 + 轻量路线决策（含对上一轮建议的自我推翻）

> 日期：2026-10-11
> 用户约束：**① 主要是自己建库　② 主打便携性、轻量化**
> 前置：deliverables/两版拆分与个人客户端优先决策-2026-10-11.md
> **本文档推翻上一轮的一条核心建议**（"保留 editor_sdk、客户端自动拉起"），理由见 §1。

---

0. 一句话结论

> **便携 + 轻量这两个约束，把"替换掉 editor_sdk"从"高风险选项"变成了"唯一可行选项"。**
> 好消息是：**Office 并不是生成依赖，只是 PDF 依赖**——所以"零外部依赖生成可编辑 DOCX"是**可达成的**。

---

1. 🔴 决定性事实：`editor_sdk` 不可分发

上一轮我建议"客户端启动时自动拉起 `editor_sdk`"。**这条路在便携场景下不成立**，实测证据如下：

事实	证据
`editor_sdk` 是**宿主应用提供的本地 MCP 服务**	`edsdk.py` 开头自述："本地 editor_sdk MCP 服务的**轻封装（HTTP 直连 POST /mcp）**"
`edsdk.py` 属于 **WorkBuddy 插件**，不在本项目	路径：`.../plugins/cache/workbuddy-builtin/skill-tencent-local-office-edit/<版本>/edsdk.py`
本机此刻**没有监听 39099**	`netstat` 查 39099/3909x 无结果 → 服务按需启动，随宿主存在
两个启动脚本都**没有**拉起它	`启动题库.bat` / `start-server.bat` 中 grep 不到 `editor_sdk`/`edsdk`

推论：老师的电脑上**没有 WorkBuddy → 没有 `editor_sdk` → 导出直接不可用**。所以"自动拉起"在**便携**场景下是死路——它假设了每台目标机器都装了 WorkBuddy。

---

2. 一个关键澄清：Office 不是"生成依赖"，只是"PDF 依赖"

这是本次最重要的技术判断，它决定了便携方案**能不能成**。

现状导出链路的真实构成（实测）

```
bank/export.py
└─ 通过 editor_sdk 调用 10 个 doc_* 工具生成 DOCX：
   doc_set_document_style / doc_modify_section / doc_get_last_operable_pos
   doc_insert_page_break / doc_insert_text / doc_insert_math(★ LaTeX → OMML)
   doc_insert_image / doc_insert_paragraph_with_text
   doc_modify_paragraph / doc_insert_border

webui/config.py::detect_office()
└─ docstring 原文："用于 PDF 导出能力判断：bank.export.export_pdf 优先 Word、回退 WPS"
```

结论

能力	依赖	能否去掉
生成 DOCX	`editor_sdk`（**不需要 Office**）	☑自研可去掉
导出 PDF	Word / WPS	⚠ 可放弃或降级（非核心）
打开/查看 DOCX	Word / WPS	❌但这属**消费端**，不是生成依赖

> **关键**：Word/WPS 是老师**打开文件时**用的（教师电脑普遍具备），**不是生成文件的前提**。
> 只要替换掉 `editor_sdk`，就能做到 **"零外部依赖生成可编辑 DOCX"**。

---

3. 轻量化的最大一笔：初始库是空的

你说"**主要是自己建库**"——这一句直接砍掉了当前最大的一块体积。

组件	当前	便携版
内嵌 Python 运行时（精简）	—	~15–25 MB
FastAPI + uvicorn + pydantic	—	~30–40 MB
python-docx + lxml	—	~10 MB

组件	当前	便携版
pypdfium2（预览）	—	~10 MB
前端构建产物	~1–2 MB	~1–2 MB
`bank.db`	**62 MB（预建 7,945 题）**	**几 MB（空库）** ⏎省下 ~60 MB
合计	—	**~80–120 MB**

目标定档：绿色免安装单目录包，100 MB 量级，双击即用。（作为参照：一个微信安装包约 200 MB+。所以"轻量"是能做到的。）

---

4. 必须自研的那一块：LaTeX → OMML

这是替换 `editor_sdk` 的**核心工程量**，也是唯一的真难点。

现状

`doc_insert_math(latex=...)` 由 `editor_sdk` 完成 LaTeX → OMML 的转换。

替代方案对比

方案	做法	体积	可行性
A. 自研转换器（推荐）	`python-docx` 生成 DOCX + 自研 LaTeX→OMML 写入 `<m:oMath>`	小（+几 MB）	☑可控
B. 内嵌 pandoc	pandoc 原生支持 LaTeX→OMML	**大（100 MB+）**	⚠ **与"轻量"直接冲突**
C. latex2mathml + `MML2OMML.XSL`	MathML → OMML 用 Office 自带 XSL	小	❌**依赖 Office，等于没解决便携问题**

建议 A。理由：项目已有反向能力（`pipeline/omml.py` 做 OMML→LaTeX），映射关系是已知的；且 LaTeX 子集有限（本项目实际用到的符号可枚举）。

⚠ 必须重踩的坑（项目历史教训，自研时会全部遇到）

坑	后果
`<a:graphicData>` 缺 `uri` 属性	**WPS 容忍，Word 报错打不开**
TAB 控制字符（`code=9/11`）	桥接拒绝 → 整卷导出中断（曾波及 1,393 题）

坑	后果
HTML 实体残留（`\Delta`→`Δ`）	公式显示错误
显式制表位停靠点	选项对齐错乱
题号双重渲染	导出件出现两套题号

> **建议**：自研前**先写探针**（参考项目 `spike/` 的做法），用最小样例验证 OMML 写入 + Word/WPS 双端可编辑，**不要在生产代码上试错**。

---

5. 便携形态定义

维度	要求
安装	绿色免安装：单目录、**不写注册表、不写 AppData**
数据位置	**相对程序目录**（现状 `outputs/bank.db` 已是相对 `ROOT`，天然便携）
U 盘	可直接放在 U 盘运行（注意 U 盘读写速度，避免频繁小文件 IO）
备份	一键"导出整库"/"恢复整库"（便携场景下换机是常态）
首启	空库 + "导入我的第一份旧卷"引导
网络	完全离线可用（不联网、不登录）

---

6. 优先级重排（便携 + 轻量版）

P0（全部精力）

编号	需求	说明
P-12	**绿色免安装打包 + 空库初始化**	定义"个人客户端"本身；目标 ~100 MB
P-00	**导出零依赖化（自研 DOCX 生成）**	⏎**本轮新增的核心工程**；去掉 `editor_sdk`
P-05	**DOCX 导入入口**（拖拽即入库）	自己建库 → **这是唯一的建库途径**，比上一轮更关键
P-01	**端到端净时间实测**	个人场景没有组织推动，**只有真省时才被持续使用**

P1

素材标注覆盖率、元数据维度、难度检索、版式图示保真、答案覆盖补录

降级 / 不做

项	处置
PDF 导出	降级为可选（需 Office）；**不阻塞便携**
P-09 贡献者署名	不做（单用户无意义）
线上部署版全部（认证/并发队列/数据分发/审计）	**明确推迟**
云端同步	不做（会重新引入账号/鉴权/冲突合并）

---

7. 风险

风险	说明	应对
🟥 **自研导出的版式保真**（最大）	现有链路已被 Word/WPS 双端实测验证；重写要重踩全部坑	**先写探针**验证最小闭环，再决定全量重写
打包体积失控	PyInstaller 常把无用依赖打进去	显式排除；实测体积作为验收项
杀毒软件误报	PyInstaller 单文件包常见	优先"目录模式"而非单文件；必要时做签名
首次建库门槛	"自己建库"意味着老师要自己导入全部资料	P-05 必须做到"拖一个 DOCX 就行"；提供空库→首题的最短路径
Office 缺失	少数机器无 Word/WPS	生成不依赖 Office ☑ 仅在"打开"时提示

---

8. 建议的下一步：一个最小验证（一次投入，四个答案）

在**一台干净 Windows 机器**上（**不装 WorkBuddy、不装 Office**，只装 WPS）：

```
解压客户端 → 双击启动 → 拖入一份 DOCX → 检索 → 组卷 → 导出 DOCX
↓
用 WPS 打开，双击公式改一个字符 → 保存 → 重开
```

这一步同时验证四件事：

1. **打包可行性**与**实际体积**（是否 ≤ ~120 MB）；

2. **无 `editor_sdk` 能否导出**（自研生成器是否成立）；
3. **无 Office 能否生成**（确认 Office 只是消费端）；
4. **便携性**——同一份包拷到第二台机器是否照常工作。

> **只有这四件事都通过，"便携 + 轻量"才算立住。** 在此之前，不建议投入任何其他功能开发。

附：与上一轮结论的差异对照

议题	上一轮结论	本轮结论（因便携+轻量）
导出链路	**保留** `editor_sdk`，客户端自动拉起	**推翻** → 自研替换（`editor_sdk` 不可分发）
P-00 定义	导出零配置（自动拉起依赖）	导出**零依赖化**（自研生成）
体积	未评估	**目标 ~100 MB**；初始空库省下 ~60 MB
PDF 导出	视为能力之一	**降级为可选**（需 Office，不阻塞便携）
线上版	推迟	推迟（不变）